Cluster and Cloud Computing (COMP90024)

Assignment 2: Big Data Analytics on the Cloud

Social Media and Parliamentary Data Analysis

Team 57

Josh Childs (1549150) Enzo Noel (1675346)
Molly Stow (1356089) YU-WEI LIN (1537312)

21 May 2025
Faculty of Engineering and IT
The University of Melbourne
Australia

GitLab Repo: `https://gitlab.unimelb.edu.au/JCCHIL/comp90024_team_57`
Video: `https://`

Scenarios

Overview Today, it is nearly impossible to spend time online without witnessing a discussion about politics. For many people, this is not a topic that is discussed politely, calmly and with respect. A thread on a hot topic quickly can become intense, escalating into arguments and attacks.
But is it truly like this everywhere, or are we just seeing the worst side of the internet?
Are there still sides of social media where people present their political opinions in a pleasant and happy way? And where does all this anger come from? Are Australians egged on by high valence political speeches? Do some politicians and parties use more aggressive language, and does this align with the discussions they spark?
In this project, we set out to answer these questions by analysing political discussions held on two different platforms: Bluesky and Reddit. We thought it would be interesting to compare the sentiment and topic of these discussions to those of political speeches being held in parliament, as provided through OpenAustralia.

Analysis Goals

Our analysis examined four key aims to understand political discussion across social media platforms and parliamentary debates.

Comparing Topics We wanted to identify which political topics are most frequently discussed across platforms. Do users on these social media sites focus on similar issues, or do they prioritise different concerns? How do these areas of discussion align with what was debated in parliament? We investigated topics such as *housing*, as well as political entities including politicians (e.g., *Pauline Hanson*), geographic locations in Australia, and political parties such as *Labor*.

Comparing Overall Sentiment We aimed to examine how the overall sentiment of discussions varied over time. Were discussions on some platforms generally more positive or negative than others? Did any clear trends emerge, potentially reflecting shifts in public mood or the political environment?

Comparing Sentiment by Subject We considered whether sentiment changes based on the subject of discussion. For example, do Reddit users express more positive views on *housing* compared to Bluesky users? What are the sentiment differences for particular parties or politicians, and how do these sentiments compare between online discussions and parliamentary debates?
Example analysis: Average sentiment by topic and for discussions mentioning specific individuals or parties.

Comparing Topics Over Time We investigated whether online discussions react to recent parliamentary debates. Does public attention to a topic increase immediately after it is debated in parliament? Does the intensity of a debate correspond with a shift in online sentiment?
Example analysis: Time series graphs of posts and debates for a topic (e.g., *housing*), showing dates of debates and volume of related online discussion.
Note: This analysis may be limited by the relatively small number of recent parliamentary debates compared to the larger volume of social media posts.

System Architecture

Overview

The system is designed to gather data from three different platforms: Bluesky, OpenAustralia and Reddit. The Bluesky client streams data continuously into elastic search using a combination of kubernetes and fission. The OpenAustralia client is made up of a number of a couple of different fission functions which

query the Open Australia API on a time trigger. Similarly the Reddit client pulls data from the Reddit API across a predefined set of subreddits. Data from the Open Australia and Reddit clients is added to redis queues, which is then processed and indexed to ElasticSearch. Analysis is performed by fission functions for sentiment analysis and named entity recognition, and the result of any analysis is cached in elastic search for future requests. User interface fission functions aggregate results from the analysis functions. The front end is a Jupyter Notebook, which queries the user interface fission functions using a RESTful API, and displays the data returned. The system is designed to be modular and scalable, so that we can easily add new platforms or data sources in the future and handle varying amounts of data. A high level overview of the architecture is show in Figure 1.

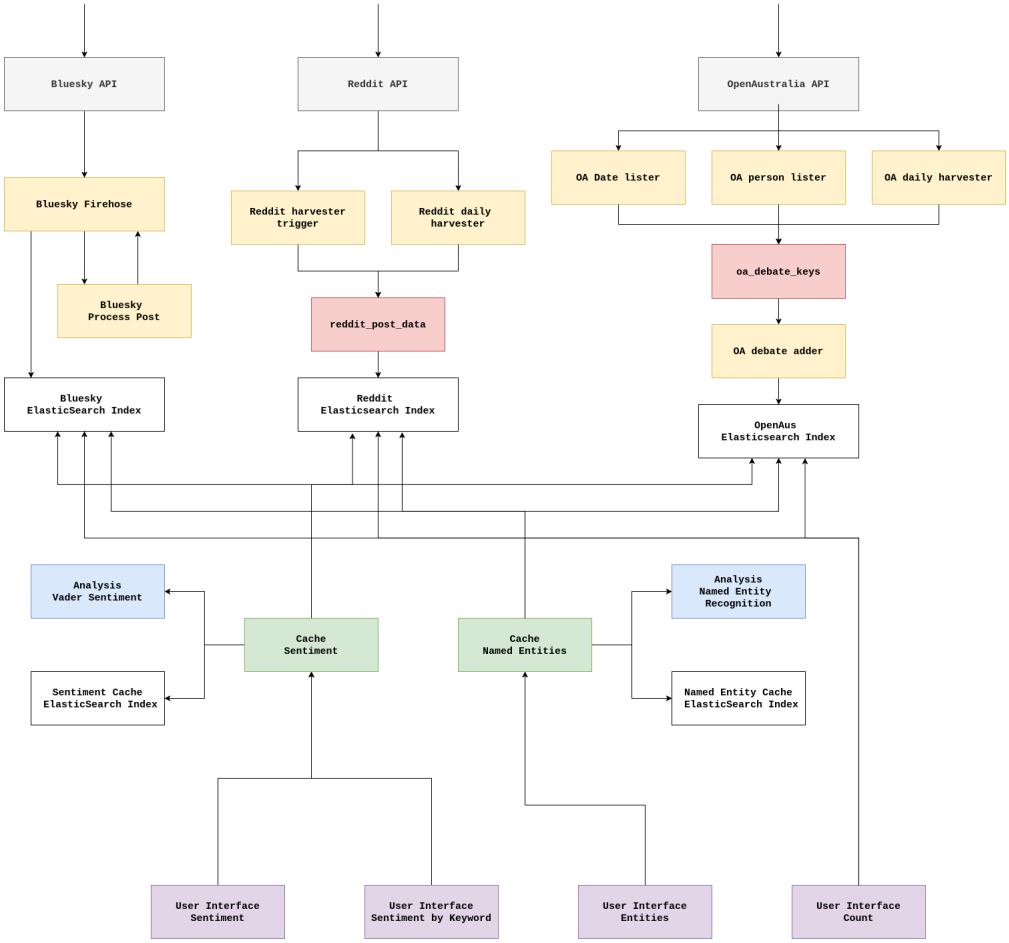

Figure 1: System Architecture. White rectangles are Elastic Search Indices, Grey rectangles are external data sources and colored rectangles are fission functions.

Bluesky

Bluesky has a firehose which gives access to all changes on the bluesky network. To handle the data from the firehose, we split the processing into two parts. The first part is receiving data from the firehose, which is written in Go and containerized in a docker image. The docker image is deployed to kubernetes, with a single replica so kubernetes will restart the container if an error occurs. The second part is decoding and extracting the useful information from a repository commit. The AT protocol sends data in blobs, which needs to be decoded. This is handled by a fission function, which receives a POST request containing the repository commit, and returns a JSON of the data we want.

The container begins by opening a websocket to the firehose endpoint `com.atproto.sync.subscribeRepos`. When we receive a `RepoCommit`, we call the fission function `processPost` which handles decoding the message. A commit could be a like on a post, a new post, a repost, or some other type of update. `processPost` returns either a 200 with a JSON of the post which contains the `cid`, `did`, `text` and `createdAt` fields, or a 204 if the commit was not a post. Bluesky uses `cid` as the unique identifier of a commit, so we reuse this in the elastic search index as the id. `did` is an identifier for the author of the post, `text` is the body of the post and `createdAt` is the post timestamp. The response is indexed to ElasticSearch using the bulk indexer.

We index the documents to Elastic Search from the firehose container. We could have the fission function index documents to ElasticSearch, but then each function would need to obtain a client login for each document, and index each document individually. In this case, the volumne of requests results in the ElasticSearch instance becoming overwhelmed by the number of requests, and would stop accepting requests. Using the bulk indexer for ElasticSearch reduces indexing overhead and increases speed.

Open Australia

The OpenAustralia (OA) API provide a REST API which we can use to query the Australian parliament's Hansard reports, and also some user comments.
A redis message queue is used to handle the data inflow from the OpenAustralia API. The queues are created by sending a post request to the `enqueue` function, which allows messages to be stored in "topic" queues.
The general structure of the OpenAustralia data inflow is as follows:

1. An "entrypoint" fission function is called, which adds the parameters (keys) of the request to the redis message queue `oa_debate_keys`.

   ```
   {
       "house": "senate" or "representatives",
       "date" or "person": date in format YYYY-MM-DD or person id
   }
   ```

2. The `oa-debate-harvester-by-details` fission function is triggered by this redis message queue.

   This function queries the OpenAustralia API for the debate details, and then queries the OpenAustralia API for the comments on that debate. This raw data is then stored in another redis queue `oa_debates_data`.

3. The `debate-adder` fission function is triggered by this redis message queue. The data is then processed and stored in the elastic search index `oa-debates-comments`.

   A parent-child relationship is created between the debate and the comments, so that we can query the comments for a specific debate. And also between debates and their parent topics, so that we can query the debates for a specific topic.

There are three main entrypoints to gathering OpenAustralia data.

- `oa-date-lister` - Queries the OpenAustralia API for dates in a year where debates were held.

  This uses a httptrigger route `oa-dates-with-debates`, providing the year as a parameter.

- `oa-person-lister` - Queries the OpenAustralia API for all debates held by a specific person.

  This uses a httptrigger route `oa-people-year-house`, providing the year and house (senate or represenatives) as parameters.

- `oa-daily-debate-harvester` - Queries the OpenAustralia API for all debates held on a specific date, and then queues the results for processing.

  This is accessed through a time trigger, which runs once a day.

The first two entrypoints are called manually to scrape past data from the backlog, while the third is called daily to gather new data. All of the functions needed to gather and process OpenAustralia data are contained within the `oa-debates` fission package.

Reddit

Data harvesting and processing architecture for Reddit is designed to handle both ad-hoc, parameterized scraping of Reddit data and a regular daily collection of recent posts. The system uses a Fission serverless workflow and Redis message queues to orchestrate smooth data flow from the Reddit API to Elasticsearch. The Reddit data pipeline, among other things, consists of the following major Fission functions and components:

1. **Entrypoint Functions (Initiating Data Collection):** There are two main Fission functions that act as entrypoints for Reddit data collection:

   - **HTTP Triggered Harvester (`reddit-harvester-new-r`):** This function gets started via an HTTP POST request through the route `/reddit-new/scrape/`. It is defined by the `reddit_harvester.py:main` entrypoint. The function expects a JSON payload specifying:
     - `keywords`: A list of keywords to search for.
     - `subreddits`: A list of subreddit names.
     - `limit` (optional, default: 10): The maximum number of posts to retrieve per keyword/subreddit.
     - `scrape_from` (optional, default: "2000-01-01"): A start date (YYYY-MM-DD format) for fetching posts created after this date.
     - `scrape_until` (optional, default: current date): An end date (YYYY-MM-DD format) for fetching posts created up to this date.
     - `sort` (optional): The sorting method for Reddit search results (e.g., "new", "top", "relevance").

     The `reddit_harvester.py` module uses the PRAW library to query the Reddit API and search for posts matching the supplied criteria. It also extracts top-level comments for each of the retrieved posts using a helper function named `get_post_comments` The posts and their corresponding comments are then packaged as JSON objects. These JSON objects, representing a list of posts and comments, are added to a Redis message queue called `reddit_posts_queue` using the utility function `util.py:enqueue_data`, which performs the enqueue operation via an HTTP POST request to a generic Fission `enqueue` function.

   - **Timer Triggered Daily Harvester (`reddit-daily-job`):** This function is automatically executed once a day, triggered by a Fission timer configured with a `@daily` cron schedule. It initiates the `reddit_daily_harvester.py:main` entrypoint. The script searches for posts through the Reddit API by referencing a hardcoded list of subreddits (`DAILY_SUBREDDITS`) and keywords (`DAILY_KEYWORDS`) within the script. It focuses on posts created within a specific recent period defined by `DAILY_SCRAPE_DAYS_AGO` (e.g., 1 day by default, but configurable as per the script's constants). The constants `DAILY_LIMIT_PER_KEYWORD_SUBREDDIT` and `DAILY_SORT_METHOD` are also mutable. It operates in a manner similar to the HTTP harvester, retrieving posts (and possibly their top-level comments via `get_post_comments_for_daily`) (depending on some configuration there). The JSON-formatted data is then sent to the `reddit_posts_queue` using `util.py:enqueue_data`. Hardcoding these subreddits and keywords in the daily harvester is deliberate, to facilitate daily scraping in an automated fashion; very well might be put into the `shared-data` ConfigMap for more flexibility.

2. **Data Queuing (Redis):** Both harvesting functions output their collected and formatted data (typically JSON strings representing batches of posts/comments) to a central Redis list acting as a message queue named `reddit_posts_queue`. Address for the Redis instance used by the Fission Message Queue Triggers is: `redis-headless.redis.svc.cluster.local:6379`.

3. **Data Indexing into Elasticsearch (Adder Function):** The `reddit-to-es-new-r` Fission function, with its entrypoint at `reddit_to_elasticsearch.py:main`, is responsible for consuming data from the `reddit_posts_queue`. There is a Fission Message Queue Trigger (MQT) of `mqtkind keda` watching `reddit_posts_queue`. When new messages (batches of posts/comments) get enqueued, KEDA starts

up `reddit-to-es-new-r`. The function loops over posts or comments in the passed batch. For each item, it checks for any document with the same `post_id` (acting as the unique identifier for both posts and comments) already present within the `reddit` index in Elasticsearch, to avoid duplicates. If the item does not exist within this index, it gets indexed within the `reddit` index. The mapping for the index is contained within `reddit-index.json`. If processing of the message fails after the configured number of retries (say 3 retries), the troublesome message is put onto an error topic/list in Redis named `reddit_adder_error` for further analysis and debugging.

This multi-tier, queue-based architecture allows maximized decoupling of data harvesting and data indexing, with resilience and scalability features. All Fission functions for the Reddit pipeline reside within the `reddit-harvester-new-r` Fission package and use the `shared-data` ConfigMap for their configurations (such as API keys, Elasticsearch connection details, and so on).

Sentiment Analysis

Choosing an Analysis Method In order for us to perform sentiment analysis on the posts from the various social media platforms we whose, we had to choose between different avaiable options. However, since we didn't want to reinvent the wheel and risk having analysis of lesser quality we decided to use a pre-existing library. Now the choice of library came down to ease of use and overall consistency with our project.
This is why we decided to use the `VADER` library. VADER is a lexicon and rule-based sentiment analysis tool that is specifically tuned to sentiment analysis in social media posts and text. It is capable of detecting sentiment in text, and it is particularly effective for short texts such as tweets or posts.

Putting it into Practice For our project, we decided to keep the sentiment analysis simple and only use a couple of values from the vader sentiment analysis function which are the following: positive, negative and neutral.
The next step was of course to implement the analysis function in our code and make it available for other functions to use. This was done by creating a new fission function called `sentiment-analysis` which takes in a string and returns the sentiment analysis values. We had to manage the dependencies and make sure that the fission function had access to the `VADER` library. This was done by creating a fission package.

Example

"The Liberal Party is not a party that is both good and bad !"

The sentiment analysis function would return the following values:

```
'neg': 0.279, 'neu': 0.462, 'pos': 0.259, 'compound': -0.1154
```

Named Entity Analysis

Why do we need Named Entity Analysis? Named entity analysis is a crucial part of this project as it allows us to identify and extract relevant subjects, persons and many others from the different posts and debates we collected.

Setting up the NER Now, setting up the NER was a bit more comlpicated as the library we chose to use `spacy` has dependencies and python version requirements and is also pretty heavy. This meant that we had to create a docker image with all the dependencies pre-built in it and then use it as an environment for our fission function.

Example

```
"text": "Barack Obama was born in Hawaii and worked in Washington."
```

The NER function would return the following values:

```
"PERSON": ["Barack Obama"],"GPE": ["Hawaii", "Washington"]
```

Analysis

Performing the analysis described above takes a non trivial amount of time. 3 simple options come to mind to handle this.

- Calculating the analysis results for all posts. Given the large number of posts we have in some indexes, bluesky in particular, this would be infeasible.
- Run the analysis on the posts queried by the user. This may take a long time if the query contains a large number of posts.
- Setup a fission function to periodically calculate and store the values for a subset of posts. This would be a better option than the two previous ones, but requires foresight into what posts are relevant, or increased effort in changing the focus.

Given the disadvantages of each, instead we implement a hybrid of 2 and 3. We calculate the values for posts queried by the user, and cache these in a separate elastic search index. When the user queries those posts again, we pull values from the cache index instead of using the analysis functions. This process is described in Algorithm 1. To increase throughput, we send the cache function a list of post id's to retrieve, reducing the overhead of authenticating elastic search clients.

Algorithm 1 Caching Analysis Results

```
Ids ← [id_0, ..., id_n]
Index ← index                                   ▷ ElasticSearch index containing the id's
Field ← field                                   ▷ Field which is being analysised
CacheEntry ← queryESForIds(Ids, cacheName, Index)   ▷ Fetch any entries from the elastic search
                                                    cache index which match the index and
                                                    any id in Ids
for id in Ids do
    if id not in CacheEntry then
        text ← queryES(Index, id, Field)        ▷ Query Index for the document with id id
                                                  and retrieve the field Field
        value ← queryAnalysisFunction(text)     ▷ Compute the value to cache
        INSERTES(cacheName, Index, id, value)   ▷ Insert the value to the cache index
        APPEND(CacheEntry, {id, value})
    end if
end for
return CacheEntry
```

User Interface

The user interface consists of Jupyter Notebook, and which access data through a RESTful API, backed by fission functions. We have 4 fission functions which provide data to the user,

- `/ui/sentiment/keyword/:keyword/start/:start/end/:end` Aggregates the sentiment per day from the start date to the end date, for each platform, filtering posts by a keyword.
- `/ui/sentiment-averager/type/:type` Calculates the average sentiment for posts containing a keyword, for each platform.
- `/ui/counts/start/:start/keyword/:keyword` Calculates the number of posts containing a keyword, per day since the given start date, for each platform.
- `/ui/named-entities/label/:label` Finds the named entities belonging to a given label, across all posts per platform. This label is a property of Spacy named entity recognition, such as 'PERSON' for a person or 'ORG' for organisations.

Some functions call other fission functions to retrieve data, such as sentiment or named entities, or they query ElasticSearch indices for documents. The function aggregates the data into a JSON object and returns it to the user, which is then visualised in the Jupyter Notebook. We use MatPlotlib for most of the visualisations, and the WordCloud python package for creating word clouds of named entities.

Fission

In alignment with the principle of declarative application management, we used fission specs to define the desired state of our fission setup.
This allows us to easily manage the state of our system, add new functions and edit the configuration of existing functions. It also means that our infastructure is "self healing", as fission will automatically re-deploy any functions which are not in the desired state. This was espeically important when working in a shared environment with a large number of group members.

NeCTAR Research Cloud

Platform Discussion

Benefits A benefit of the NeCTAR Research Cloud and fission is the ability to use multiple programming languages. Due to issues with python, we switched the Bluesky harvester to Golang. Additionally the analysis caching was written in Go, the main benefit of using Go is the easy concurrency for requesting new analysis results. Team members not familiar with Go could still use the cache function since it could be accessed as a fission function from Python code.
The NeCTAR Research Cloud simplified the development of the application, by giving a homogeneous environment to develop the application. We did not need to be concerned about what operating system the end user or other team members are using, if they are using the same version of python, or other tools, and other details.

Challenges A major challenge our group faced was adapting to the shared nature of the NeCTAR research cloud. At several points during the project, we encountered issues arising from multiple people attempting to modify the system at once.
Some of the minor issues included conflicting fission deployments, or source code being removed without the fission yaml specs being removed, resulting in a spec that could not be deployed. However, we also encountered more serious issues. Near the beginning of the project, the cluster was deleted and the setup was unexpectedly re-run, meaning that we had to re-build our software stack. And not long after the cluster was re-setup, a duplicate ElasticSearch deployment was made to the default namespace, alongside the ElasticSearch deployment in the elastic namespace. This didn't seem to break anything and was removed without affecting the project. A challenge of NeCTAR is the lack of Role Based Access Control within a group, meaning we couldn't have one person responsible for managing the cluster.
Even at a small group scale, it was difficult to coordinate the changes being made to the system. This was mitigated by the use of fission specs, as we could easily re-deploy the functions to a different worker node. These challenges highlighted the benefits of using an Infrastructure as Code approach, as we could recover from these setbacks.
Another challenge of using fission is the inability to carefully instrument programs when issues occur within a function. In this case, Bluesky would not work in python. A simple function to pull from a feed would work locally but not in fission. Tracing it back to a minimal example, having `import atproto` in a python file would cause the function to not execute at all. So we switched to using Go for the Bluesky handling.

Application Scaling

Bluesky To handle processing the large number of posts in the bluesky firehose, we utilise the fission new deploy executor to scale the number of pods used for processing posts automatically. We set the minimum number of pods to 1, and the maximum to 30. This allows fission to increase or decrease the number of pods, up to 30, depending on the current load. Since the firehose container is implemented using Go, we

spawn a Go thread for each request, allowing the firehose to scale vertically to meet demand since we can't scale the websocket horizontally.

Using grafana, we can monitor the system resources used by the bluesky firehose. At the peak, the fission function handles 35,000 requests per second, with around 5,000 of those requests containing a post, shown in Figure 2. The firehose websocket pod utilises 0.8 cpu's at the peak, and 0.4 cpu's for the fission function, shown in Figure 3.

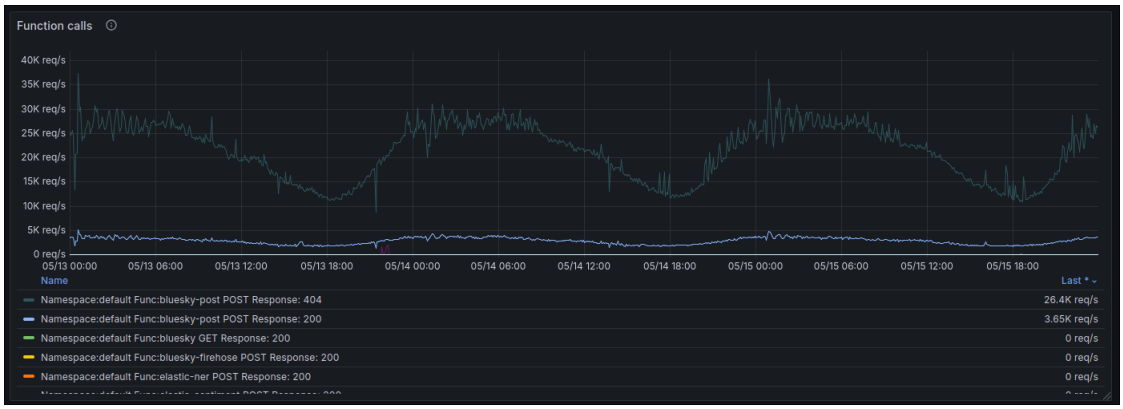

Figure 2: Bluesky fission function calls.

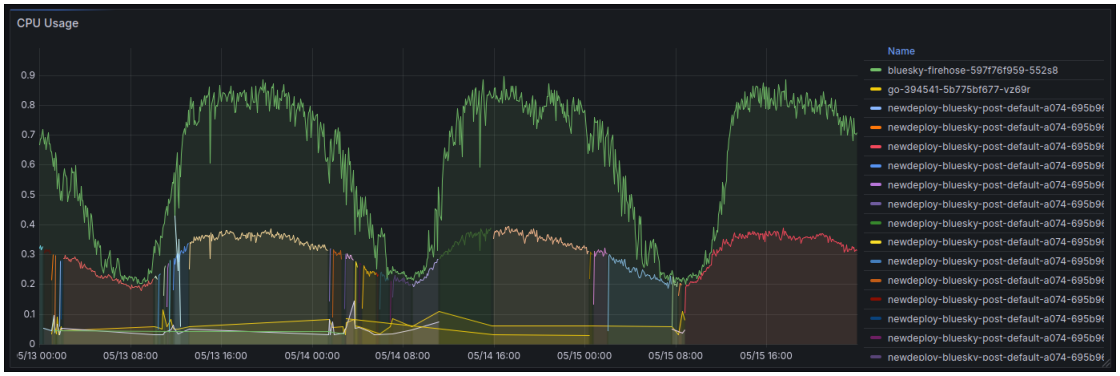

Figure 3: Bluesky cpu usage.

OpenAustralia The data influx of the OpenAustralia api was of a less intense "velocity" compared to that of BlueSky and Reddit. This meant that the regular scraping could be done on a daily basis, gathering data from debates held two days prior, with to allow complete collection of data, as the Hansard reports are finalised by 5pm the day after the debate. This was achieved using a time based fission trigger `oa-daily-debate-harvester`, which runs once a day. Similarly to the BlueSky setup, the number of pods were dynamically allocated as needed. However, due to the lower amount of incoming data the minimum number of pods was set to 0, as the resources required when starting a new pod would not significantly affect the performance.
The message queues used a Keda workload autoscaler. This monitored the number of messages in the queue, and scaled the number of pods up or down depending on the number of pods required. This helped handle the variabible load of the OpenAustralia API, as the number of messages in the queue could vary from 0 to 1000+, especially when gathering data from previous years.

As a side comment, the usefulness of the `oa-daily-debate-harvester` was limited while collecting data for our project, as The Houses of Parliament are in recess from 16 May until 25 May 2025 and so no new debates were available to add.

Reddit The Reddit harvester utilizes two serverless functions provided by Fission: one HTTP-triggered (`reddit-harvester-new-r`) and the other timer-triggered (`reddit-daily-job`) to support both manual and daily scraping. Both functions run on Fission's `poolmgr` executor, which maintains pre-warmed pods, effectively minimizing cold-start latency. These harvester functions collect Reddit posts and top-level comments, then enqueue the results into Redis (`reddit_posts_queue`) using a generic Fission `enqueue` function.
For indexing, the system employs the `reddit-to-es-new-r` function, which is autoscaled by KEDA based on the length of the Redis queue. When the number of messages exceeds 1000, KEDA scales up the number of pods for parallel processing. Conversely, it scales them down when the queue is idle. This enables efficient autoscaling in response to fluctuating data loads, such as large historical scrapes or periods of high Reddit activity.

Analysis In the System Architecture section, we discussed the cache used for analysis functions. When we don't have a cached value for a post/debate, we need to query the analysis functions to compute the values. If there is a large number of posts which are not cached, this can take a long time. To speed up calculations, we take advantage of fission and Go. If we find a value we need to calculate in the caching function, we spawn a go thread. This thread is responsible for calling the fission function for calculating the value and caching the result. This allows the cache function to calculate all missing values concurrently, while using functions from other languages such as Python. The analysis functions utilised the fission new deploy executor to handle the spikes in queries resulting from the cache function making a large number of requests at once. Additionally the cache function requires less vertical scaling compared to calculating the values in the cache function, since most of the threads will be inactive while waiting for the response from the fission function.

Error Handling

Bluesky The Bluesky firehose client is deployed as a container, instead of a fission function, since the websocket can listen indefinitely. This container is deployed to kubernetes with one replica. This means the container will be re-deployed to another worker node if one fails, and kubernetes will spin up a replica if the current one crashes. If an error occurs in the fission function which processes posts, a server error code and the error message is returned in the response. The Bluesky client logs out the response and continues on if it receives an error from the fission function.

OpenAustralia A modular, "Event Driven Architecture" approach was taken to handle errors in the OpenAustralia API. The logic associated with querying the OpenAustralia API was split into three key parts: the entrypoint functions, the harvester, and the adder.
These are connected by redis message queues, each with their own "error topics". This seperation of functionality improves the robustness of the system, as we can see exactly where any errors occurred, and retry only that part of the process. Somethiungsomething Event driven architecture It also means that if one part of the process fails, the outputs of the other functions will still be saved, allowing the process to continue without data loss.
This was particularly useful when querying the OpenAustralia API, as the formatting of responses is often unpredictable, even within the same endpoints.
The immediate httptrigger entrypoint functions do not have any retry logic, as they are only called once manually, and the data is stored in the redis queue. The `oa-debate-harvester-by-details` and `debate-adder` functions are triggered by the redis queues, and have retry logic built in. Both of these functions will retry up to 3 times if they encounter an error, and will log the error message to their respective "error topic" redis queue.
This strategy is certainly not perfect, as it does not handle all possible errors, but it does allow us to see where the errors are occurring, and retry the failed functions.

Reddit A modular, "Event Driven Architecture" was used to handle errors in the Reddit pipeline. The logic is split into three parts: the entrypoint functions (`reddit-harvester-new-r` and `reddit-daily-job`), the harvester, and the adder (`reddit-to-es-new-r`). Each stage is connected via Redis queues, with dedicated error queues (dead-letter queues) for isolating and diagnosing failures.
This design improves robustness by allowing specific parts of the pipeline to fail and retry independently. If one part encounters an error, the others can continue processing. It also makes error tracing easier, as each failure is logged to a specific error queue for offline inspection.
The HTTP-triggered `reddit-harvester-new-r` performs parameter validation and enqueues results. Since it's manually triggered, it does not include retry logic, but validation failures (e.g., malformed dates or missing fields) result in an immediate 400 error response.
The timer-based `reddit-daily-job` uses try/except blocks to handle API failures gracefully—logging errors for failed subreddits but continuing with the rest. Both entrypoints use a shared `util.py:enqueue_data` function to push messages into Redis, and failures at this step are logged but non-fatal.
The `reddit-to-es-new-r` adder consumes messages from Redis and indexes them into Elasticsearch. It uses Fission's Message Queue Trigger (MQT) with `--maxretries 3`. If all retries fail, the message is moved to the `reddit_adder_error` dead-letter queue. This prevents single bad messages from blocking the queue.
Indexing is robust: malformed JSON raises deserialization errors, individual post failures (e.g., due to bad mappings) are caught and logged, and duplicates are avoided via `es_client.exists()`.
This approach doesn't cover all possible failure cases (e.g., persistent external outages), but it provides a strong foundation for resilience and observability in a high-throughput, asynchronous system.

System Fault Tolerance and Resilience Analysis

Kubernetes Layer

Advantages

- **Pod Self-Recovery**: Kubernetes restores failed Pods automatically. This includes Fission function Pods, the Bluesky Firehose Client Pod, and other service Pods like ElasticSearch and Redis (assuming appropriate controllers like `Deployment` or `StatefulSet` are used).
- **Node Failure Recovery**: If a Kubernetes worker node fails, Pods managed by `ReplicaSet`, `Deployment`, or `StatefulSet` are rescheduled on healthy nodes.

Potential Issues/Considerations

- **Kubernetes Control Plane**: Although generally highly available, it may become a failure point in self-managed or small clusters. In research cloud environments (e.g., NeCTAR), control plane reliability is maintained by the provider.
- **Single Node Cluster**: A single-node Kubernetes cluster would suffer total downtime if that node fails.

Fission (Serverless Layer)

Advantages

- **Function Instantiation**: Fission's Executors (`NewDeploy` or `PoolManager`) manage function lifecycle. `NewDeploy` creates a Pod on demand; `PoolManager` maintains a warm Pod pool.
- **Scalability and Fault Tolerance**: Horizontal scaling (e.g., via KEDA) helps tolerate individual Pod failures. Other Pods can still handle requests.

Potential Issues/Considerations

- **Fission Core Components**: Components like the Router, Controller, and Executor are also Kubernetes Pods. If these fail without recovery, function routing and deployment break down.
- **Cold Starts**: Functions without warm Pods (especially under `NewDeploy`) may suffer from startup latency, which might seem like service unavailability. The use of `PoolManager` can mitigate this (e.g., for Reddit harvester).

Data Collection & Processing Pipeline

Bluesky Firehose Client

- **Current State**: Deployed as a Docker container, Kubernetes restarts it on crash.
- **Potential Single Point of Failure**: Single replica implies no backup during restart. Missed real-time data may be lost if not recoverable via the Bluesky API.
- **Fission `processPost` Function**: If this Go function fails or lacks scalability, it could delay or drop data.

OpenAustralia and Reddit Harvesters (Using Redis Queues)

Benefits (Event-Driven Architecture):

- **Decoupling**: Redis queues (e.g., `oa_debate_keys`, `reddit_posts_queue`) isolate data collection, processing, and indexing, preventing cascading failures.
- **Retries**: Retry logic is built into functions (e.g., up to 3 retries) and failed messages are routed to error queues (e.g., `reddit_adder_error`), promoting robustness.

Potential Issues/Considerations:

- **Redis Service Stability**: Redis failure halts the pipeline. Persistence and high availability (e.g., via Sentinel or Redis Cluster) can improve resilience.
- **Error Queue Management**: Error queues require active monitoring and remediation; otherwise, failures remain unresolved.

Data Storage & Caching (ElasticSearch)

Advantages

- **Analytical Caching**: ElasticSearch caches NER and sentiment results, avoiding recomputation and providing redundancy if analysis functions are temporarily unavailable.

Potential Issues/Considerations

- **ElasticSearch Stability**: A failure (even in single-node mode) halts indexing, cache reads/writes, and data access through the UI.
- **Data Loss Risk**: Effective persistence and backup strategies are crucial.
- **Indexing Errors**: Issues like mapping conflicts or disk space exhaustion must be mitigated with logging, retries, and alerting mechanisms. Your report mentions `reddit-to-es-new-r` handles such cases.

Analysis Functions (Vader Sentiment, NER)

Advantages

- **Fission Function Deployment**: These benefit from Kubernetes and Fission capabilities like auto-scaling and self-healing.

Potential Issues/Considerations

- **Resource Usage**: NER via SpaCy is resource-intensive and uses custom Docker images. High concurrency may overload nodes or affect other Pods.
- **Model/Library Dependency**: Bugs or inefficiencies in libraries directly impact reliability.

User Interface (Jupyter Notebook + Fission APIs)

Potential Issues/Considerations

- **API Backend Failures**: Failure in UI Fission API functions prevents Jupyter from retrieving data.
- **Jupyter Environment**: A crash or misconfiguration (either locally or in a Pod) prevents user interaction with the system.

Integration Testing

We have automated testing in the `test` directory of the GitLab repository. The script `run_tests.sh` runs the each of the tests in one script. For Bluesky we test the process post function correctly decodes a post, and return a 204 status code for a non-post. For the caching function, we test that calculated values are inserted to the caching ElasticSearch index, and if we retrieve a cached value, the value is the one in ElasticSearch and is not recalculated. For OpenAustralia, we mock the date lister and person lister functions, to ensure they pull the correct data from the API and make the correct calls to the enqueue function. For Sentiment and Named Entity analysis, we check the output of the fission functions match the expected output.

Analysis findings

For Reddit, we collected data from a collection of subreddits related to Australia. For Bluesky, we filter posts to those containing 'auspol', a common hashtag on Bluesky for discussing Australian Politics.

Topics Across Different Platforms

To compare the topics of the posts and debates, we use word clouds to show the most common named entities in the posts and debates. The python library `spacy` implements Named Entity Recognition, which we use to extract entities from text. Word clouds are generated using the `wordcloud` python library. Figure 4 shows a wordcloud of organisations discussed in posts and debates across each platform. Figure 5 shows a wordcloud of people discussed in posts and debates across each platform. Wordclouds are excel at displaying a large amount of data, and giving qualitative comparisons between different data points, but doesn't show quantitative differences in values very well. Figure 6 shows the top 10 organisations across the platforms in a bar chart.

Sentiment Over Time Across Platforms

The overall sentiment of the posts and debates was calculated using the `VADER` library. The sentiment was calculated for each post and debate, and then averaged over time. This allowed us to compare the overall averaged sentiment of the posts and debates over time. We found that this was relatively consistent across the platforms, with only small variations sentiment. Figure 7 shows the sentiment of Reddit and Bluesky from the 1st to the 19th of May 2025. Bluesky posts averaged a negative sentiment of around 7%, while

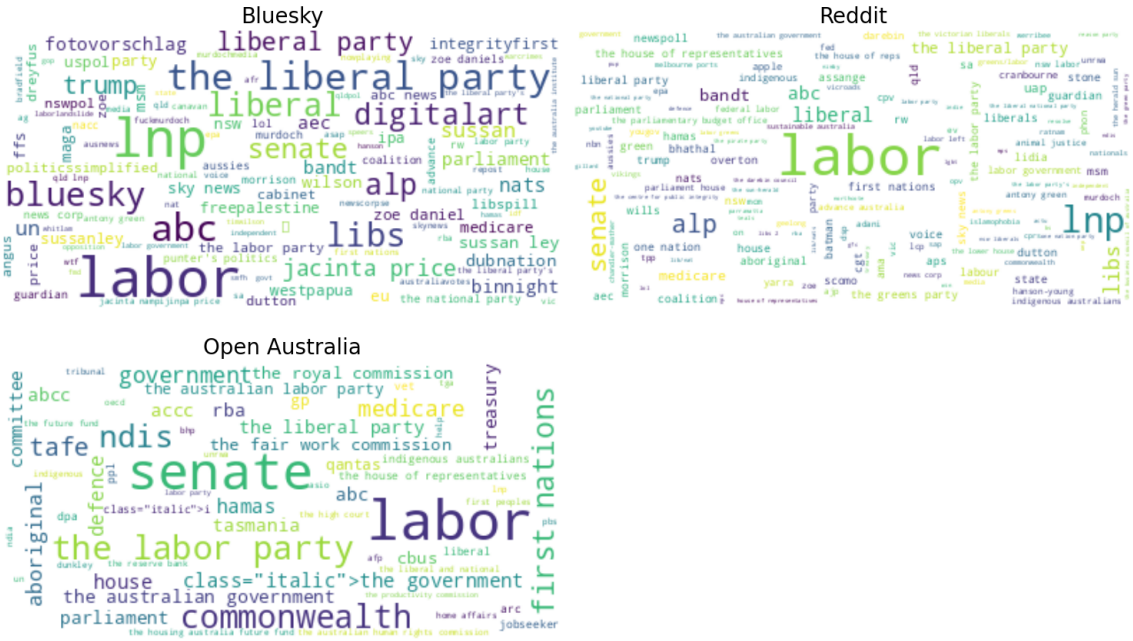

Figure 4: Word cloud of the most common "Organisation" entities in the posts and debates.

the average negative sentiment of reddit posts ranged from 5% to 9%. Both BlueSky and Reddit posts were generally mostly neutral, but slightly more positive than negative.
Figure 8 shows the sentiment across two weeks of debates in 2021. The sentiment is more positive than negative, with around 10% positive and 5% negative. As more data is collected over time, we would be able to display sentiment across all 3 platforms for the same time period.

Sentiment By Keyword Across Platforms

To show overall sentiment around topics or keywords, we average the sentiment of all posts containing a keyword. For OpenAustralia, if the topic was a person or party, the average sentiment of debates held by that entity were calculated. If the topic not a person or party, debates mentioning that topic were averaged. Figure 9 shows the sentiment of posts discussing "Pauline Hanson" compared to the sentiment of debates where "Pauline Hanson" was a speaker. Figure 10 shows the sentiment of posts discussing "Anthony Albanese" compared to the sentiment of debates where "Anthony Albanese" was a speaker.

Post Count By Keyword Across Platforms

We wanted to investigate if debates influenced online discussions. To do this, we calculated the number of posts and debates containing a set of keywords over time. For example, Figure 11 shows number of posts and debates containing the keyword 'housing' over time. Unfortunately there is limited overlap of Bluesky and Open Australia data due to the short timeframe of data collection. In the future we may see a shift in the topics of online discussions, or the amount of discussion on a topic, after a debate is held.

Conclusion

This project successfully developed a modular and scalable system for collecting, processing, and analyzing political discussions across various platforms. These platforms include social media sites such as Bluesky

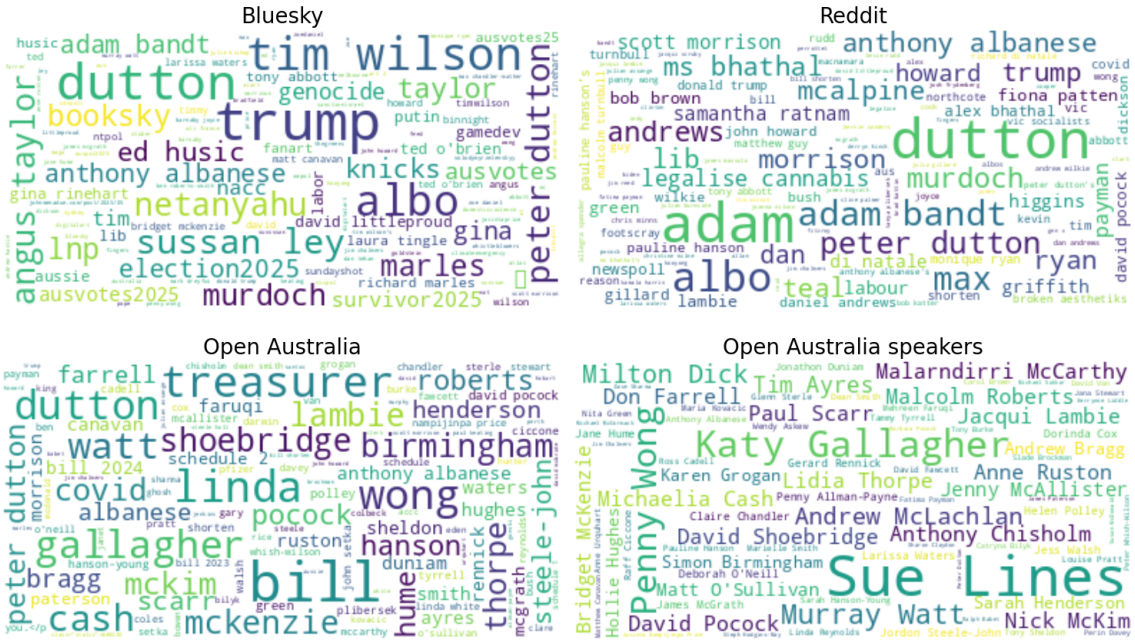

Figure 5: Word cloud of the most common "Person" entities in the posts and debates.

and Reddit, as well as parliamentary records from OpenAustralia. By utilizing cloud computing technologies—including Kubernetes, Fission, Elasticsearch, and Redis—the system enabled effective sentiment analysis and named entity recognition on large datasets.

The primary analytical goals included comparing discussion topics across platforms, tracking sentiment trends, analyzing sentiment by subject, and exploring the connection between social media conversations and parliamentary debates. Initial findings revealed that the political entities (e.g., organizations and individuals) discussed varied by platform. These differences were illustrated using word clouds and frequency charts. Sentiment analysis showed that social media content tended to be neutral to slightly positive, whereas parliamentary debates often leaned more positive. However, due to a limited timeframe and number of cases, it remains unclear how social media sentiment reflects or responds to parliamentary discourse.

Throughout the project, the team addressed various challenges inherent in the shared NeCTAR Research Cloud environment. For instance, they managed configuration issues and cluster rebuilds effectively using an Infrastructure as Code approach through Fission specs. Other technical difficulties, such as using the `atproto` Python library within the Fission environment, were also resolved. Additionally, the team implemented robust scaling and error-handling strategies—including event-driven architecture and dead-letter queues—for all major data sources.

Despite constraints like a short data collection period, the project demonstrated the system's capabilities and yielded valuable insights into the interaction between social media and political discourse in Australia. With a longer observation period and more data, future research could provide even deeper analysis of the dynamics between these platforms.

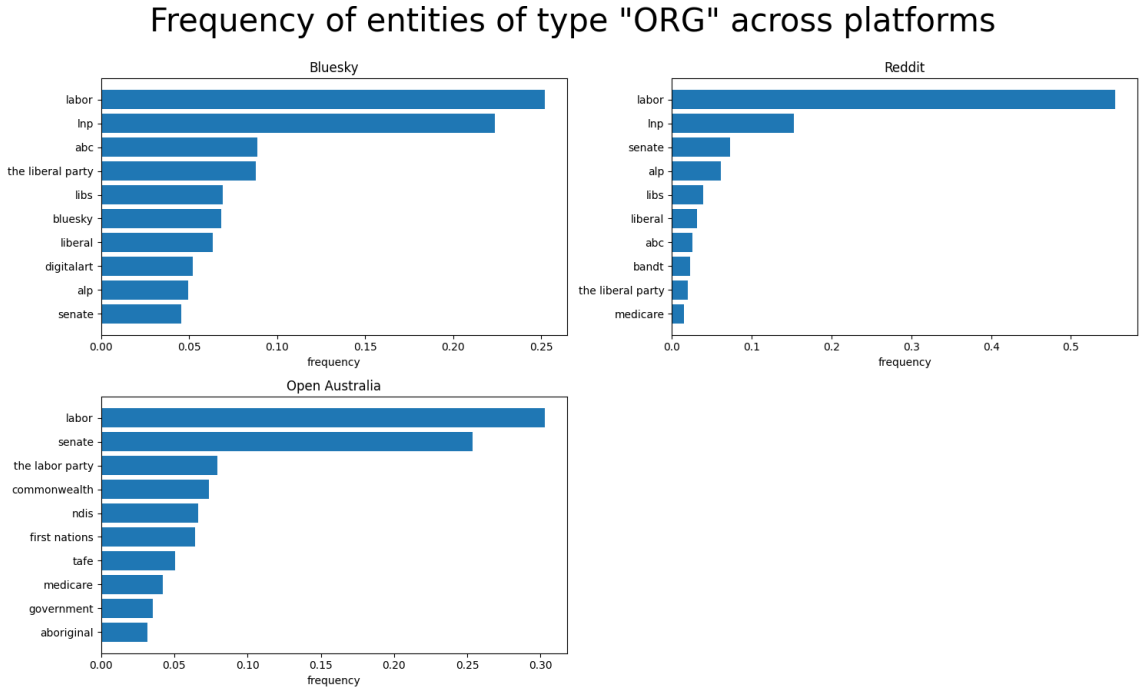

Figure 6: Top 10 most common "Organisation" entities in the posts and debates.

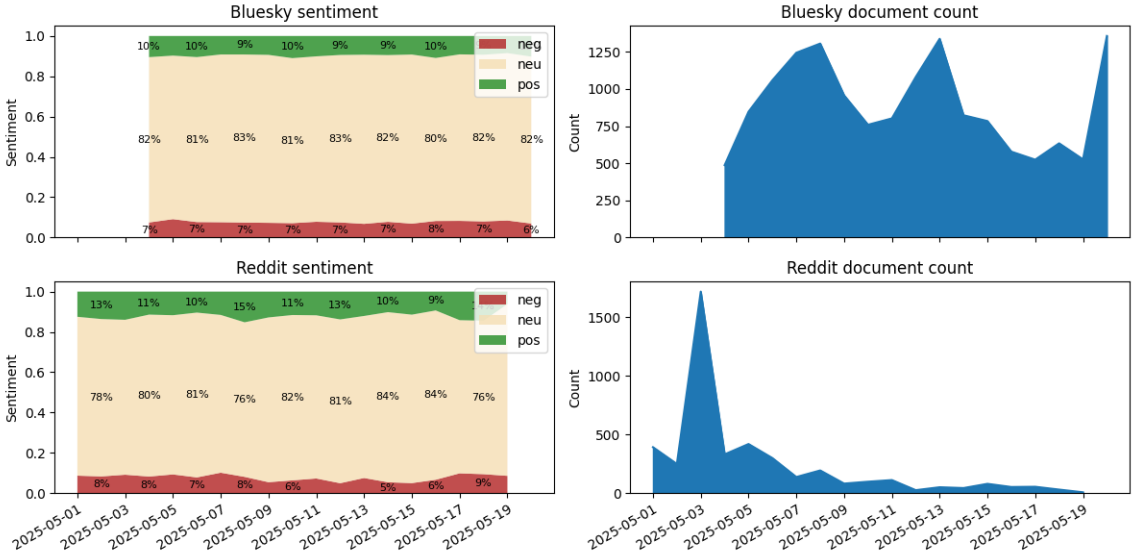

Figure 7: Sentiment of posts related to Australian politics between 1st May 2025 and 19th May 2025

Figure 8: Sentiment of debates between 22nd August 2021 and 3rd September 2021.

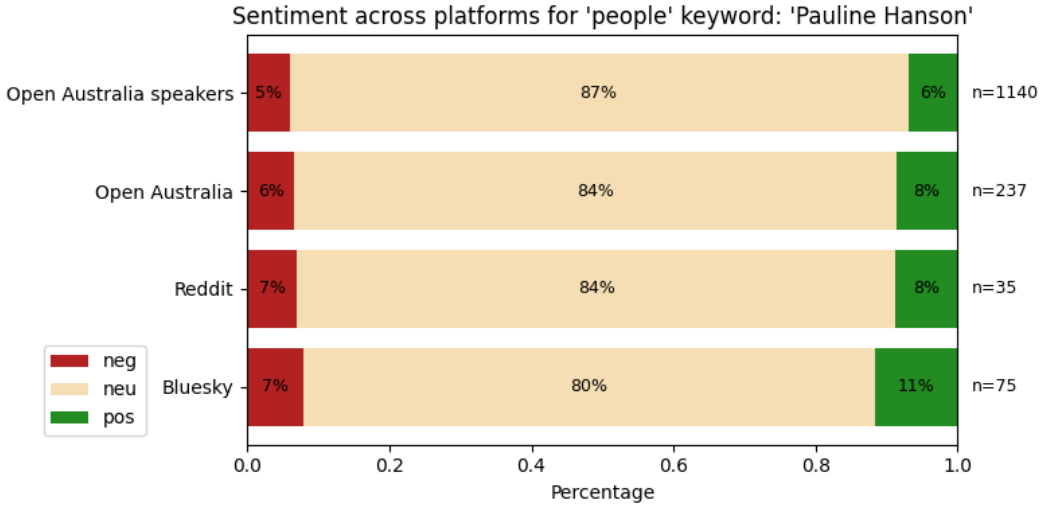

Figure 9: Sentiment of posts discussing “Pauline Hanson” and debates where “Pauline Hanson” was a speaker.

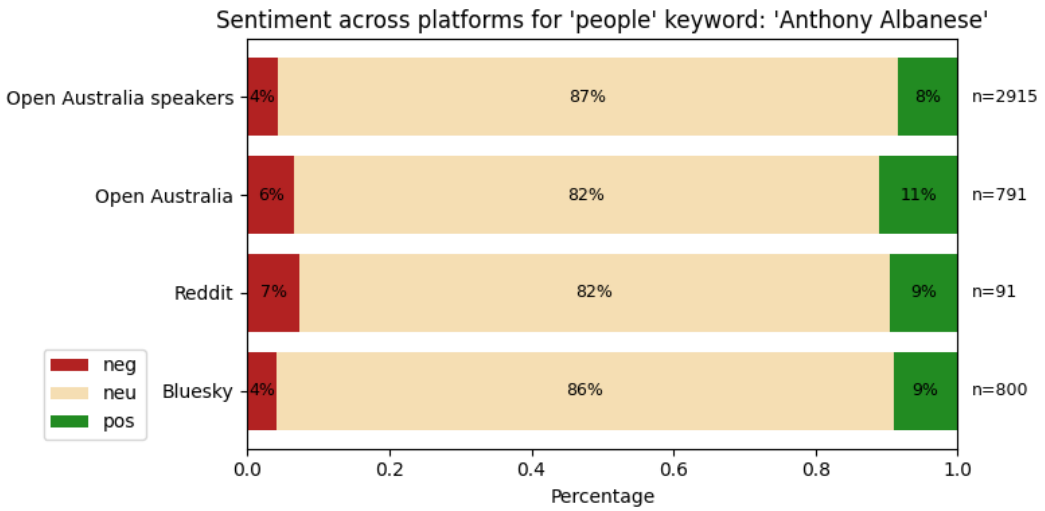

Figure 10: Sentiment of posts discussing "Anthony Albanese" and debates where "Anthony Albanese" was a speaker.

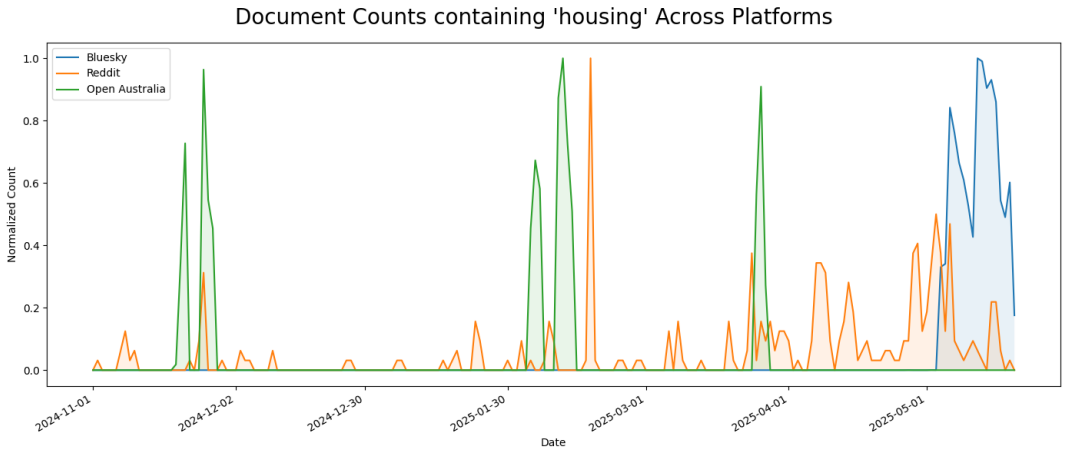

Figure 11: Frequency of posts containing the word 'housing' per day.